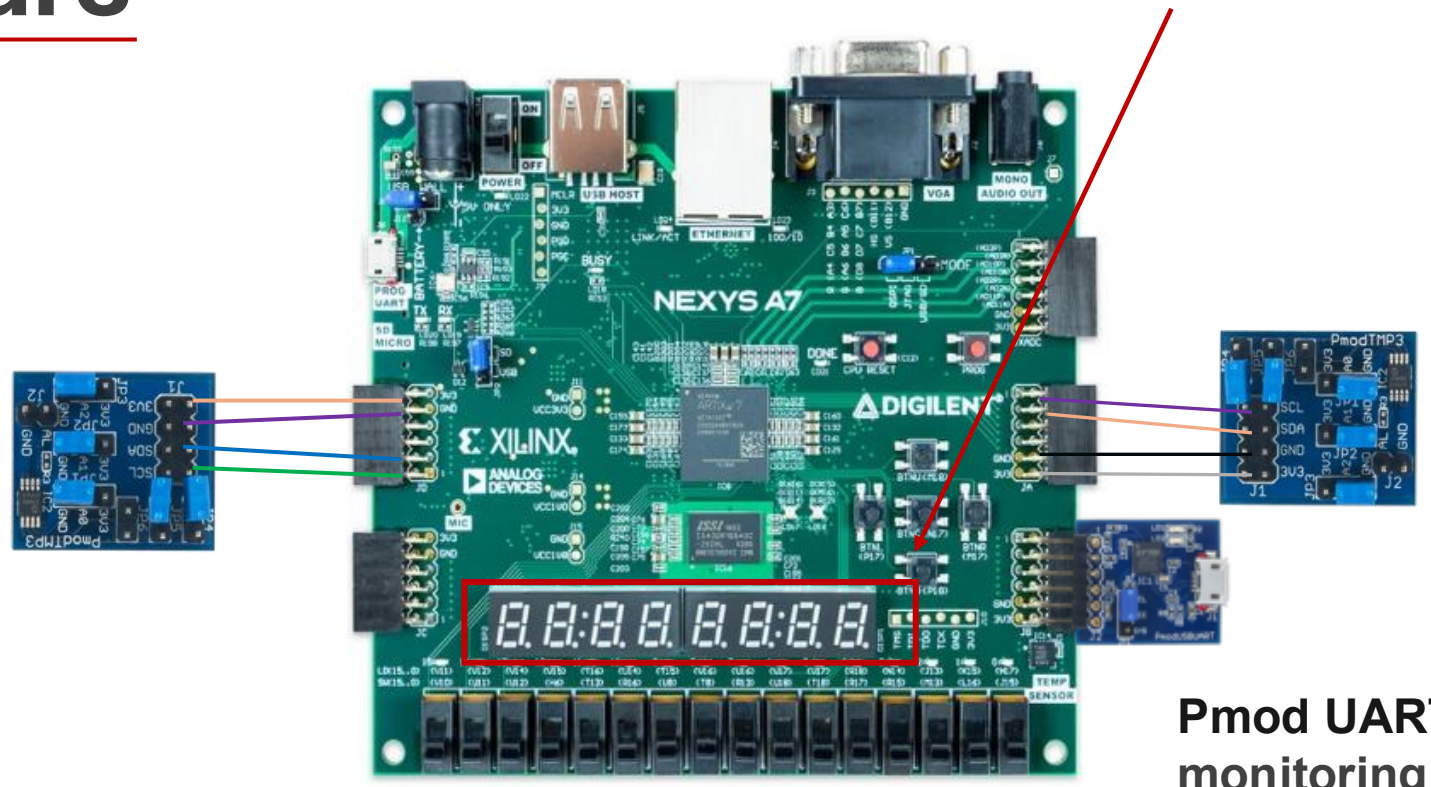
Functional Safety in Thermal Management Systems

Presented by

Maria Anthonyos
Mohamad Abdo
Jonathan Bahry

Hardware

The Seven segment display is used to indicate system states such as COOL and STOP

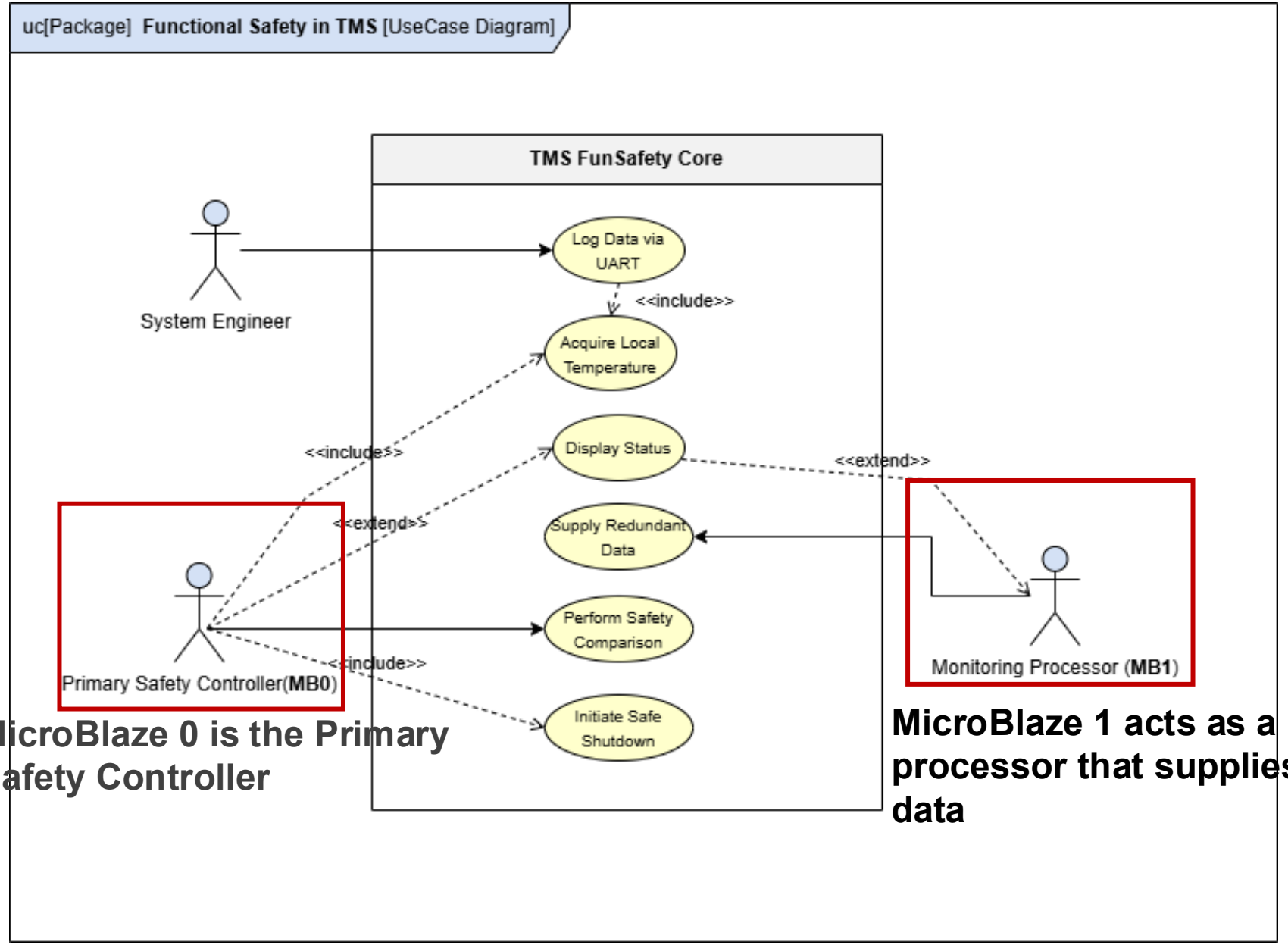


Pmod UART is used only for monitoring and debugging

The system is implemented on a **Nexys A7 FPGA** which hosts two MicroBlaze soft processors running in parallel

Each processor is connected to its own external **Pmod TMP3** temperature sensor

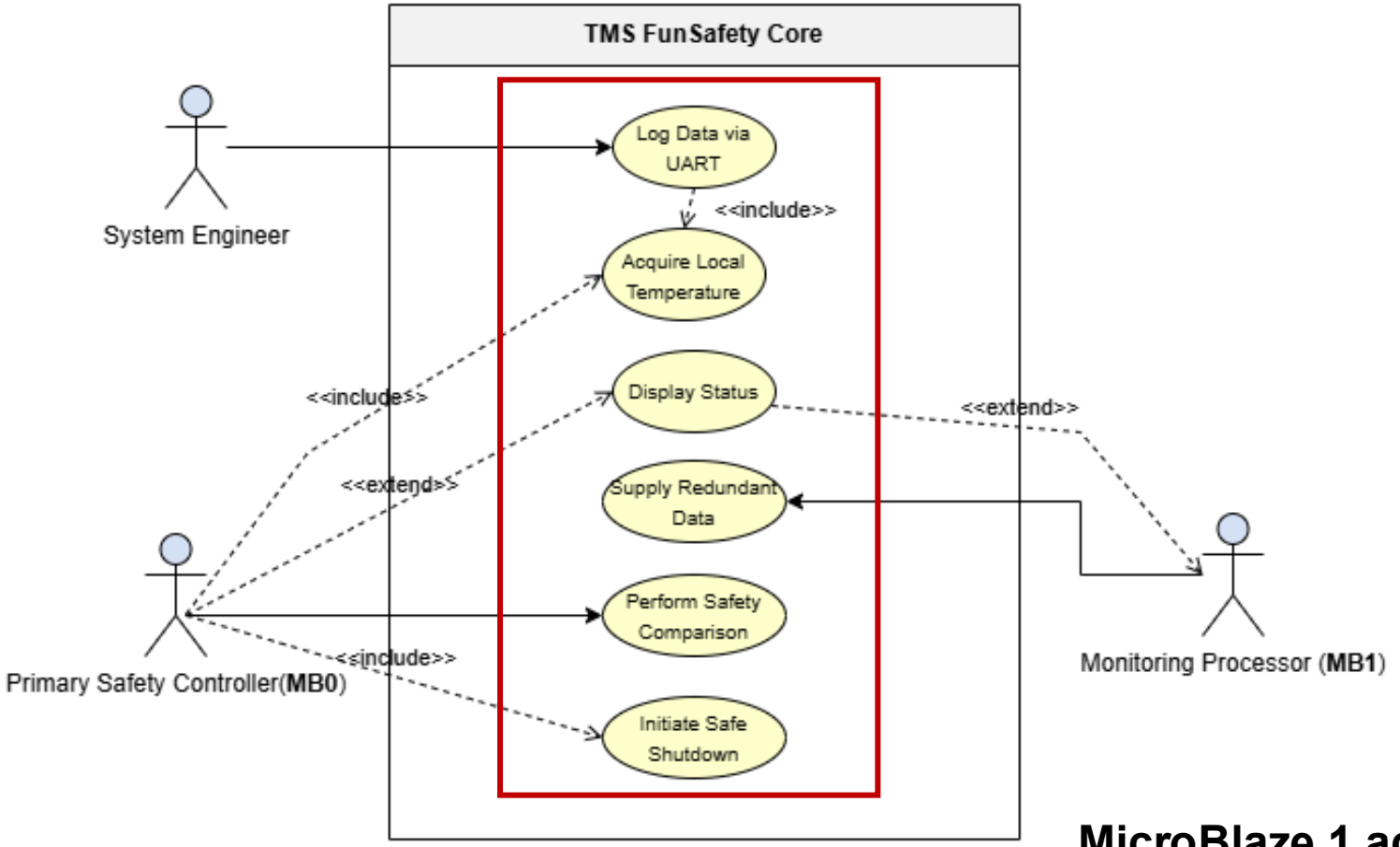
Use Case Diadram



MicroBlaze 0 is the Primary Safety Controller

MicroBlaze 1 acts as a monitoring processor that supplies redundant data

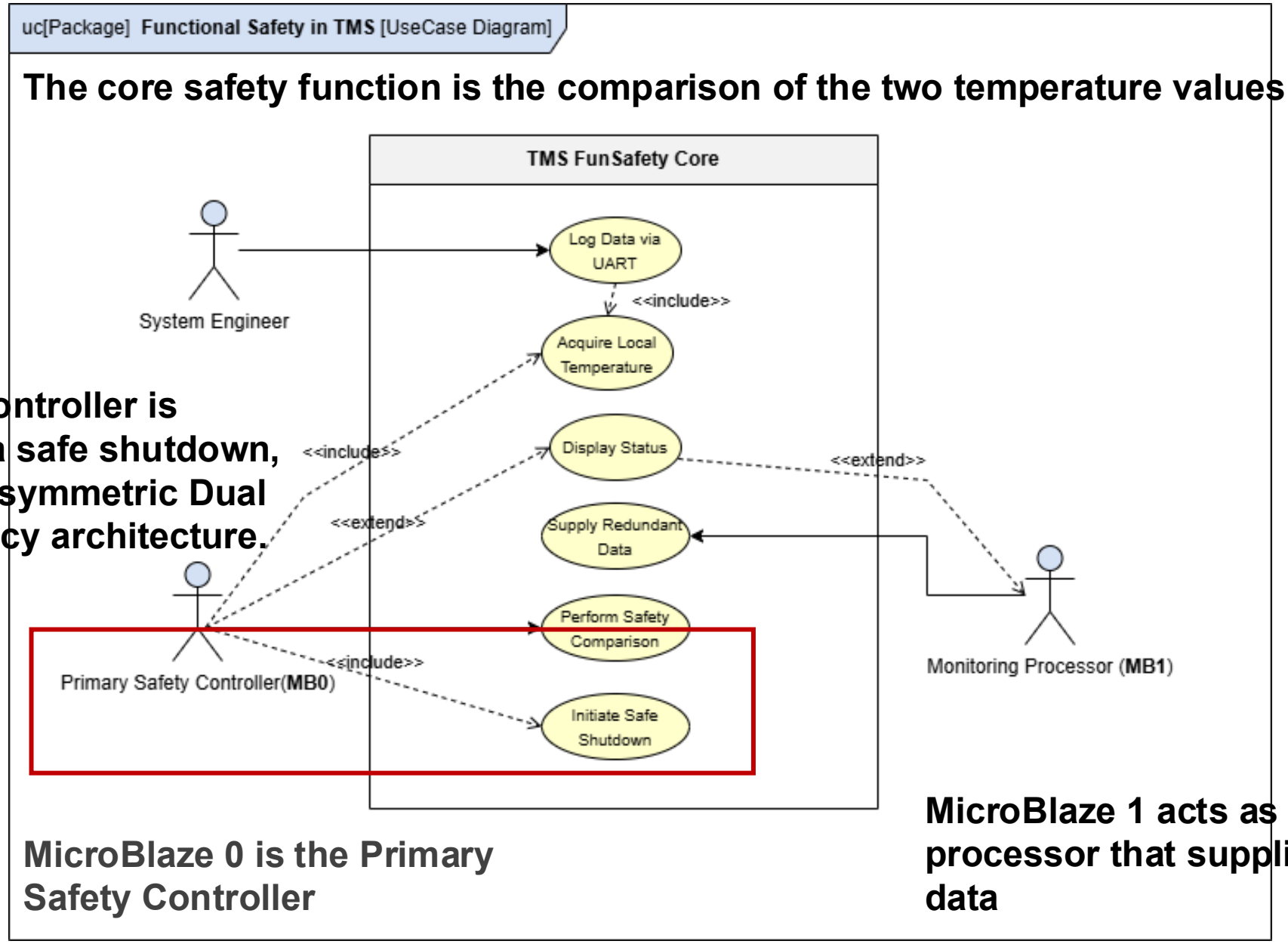
The core safety function is the comparison of the two temperature values



MicroBlaze 0 is the Primary Safety Controller

MicroBlaze 1 acts as a monitoring processor that supplies redundant data

The core safety function is the comparison of the two temperature values

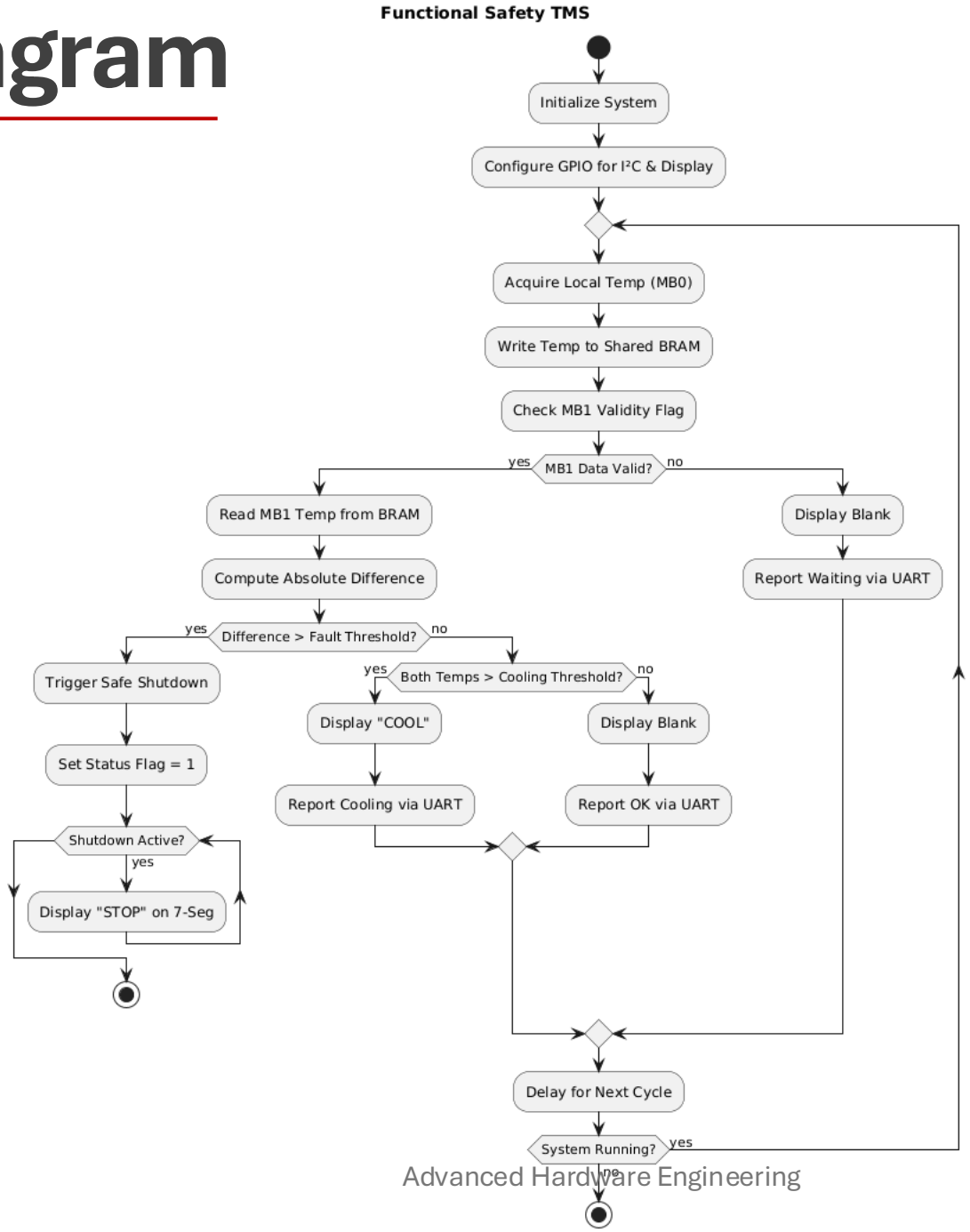


Only the primary controller is allowed to initiate a safe shutdown, which reflects an asymmetric Dual Modular Redundancy architecture.

MicroBlaze 0 is the Primary Safety Controller

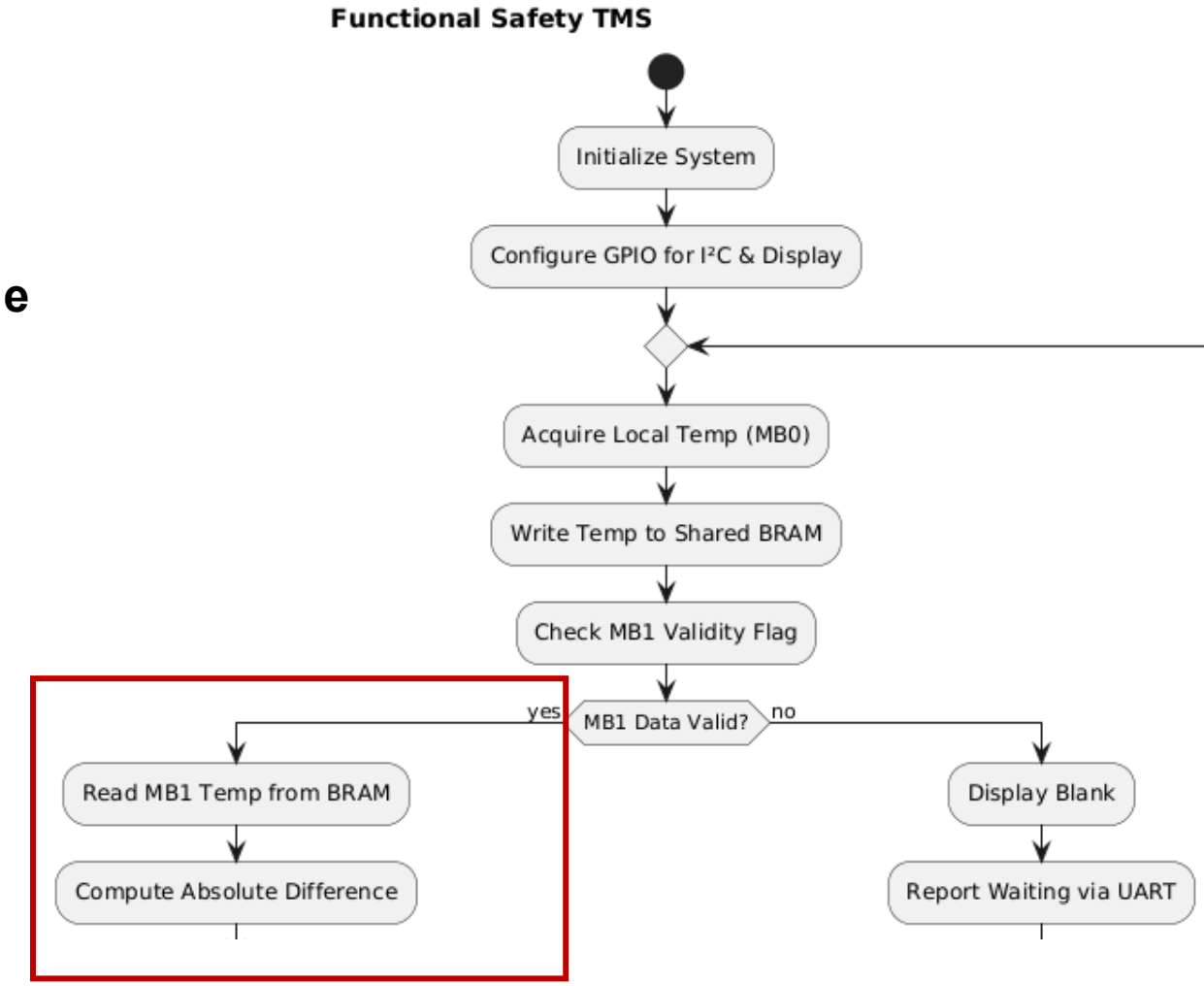
MicroBlaze 1 acts as a monitoring processor that supplies redundant data

Activity Diagram

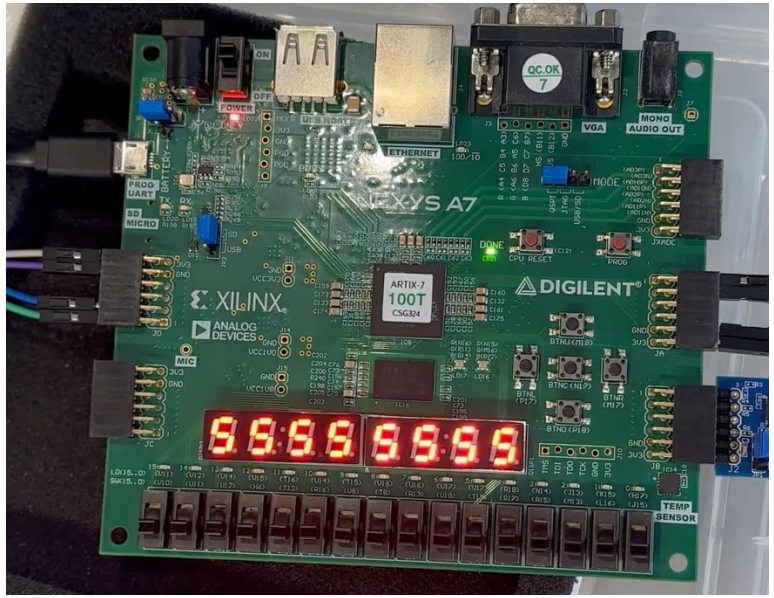


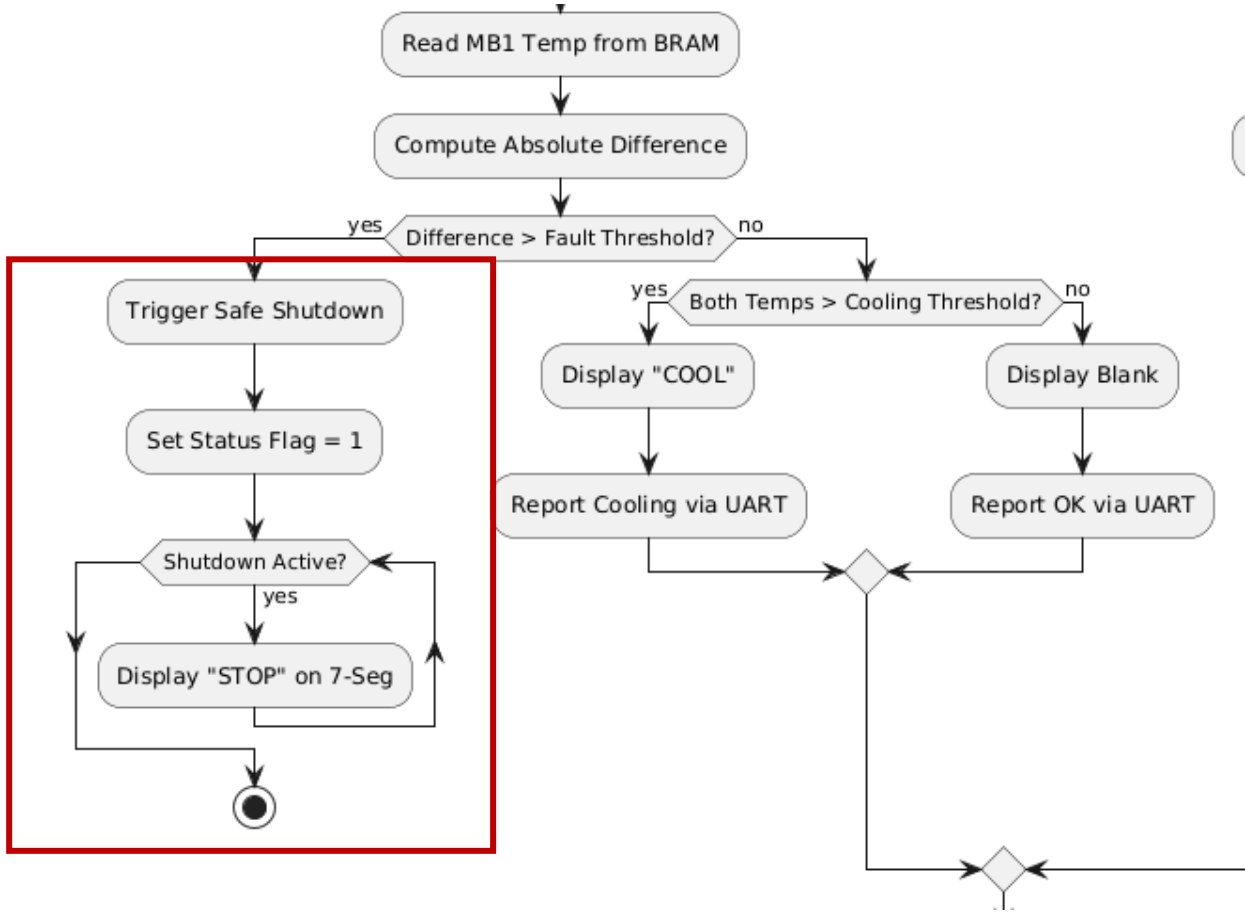
After initialization, the system continuously reads the local temperature and waits for the redundant value

Once both values are available, the system computes the absolute difference

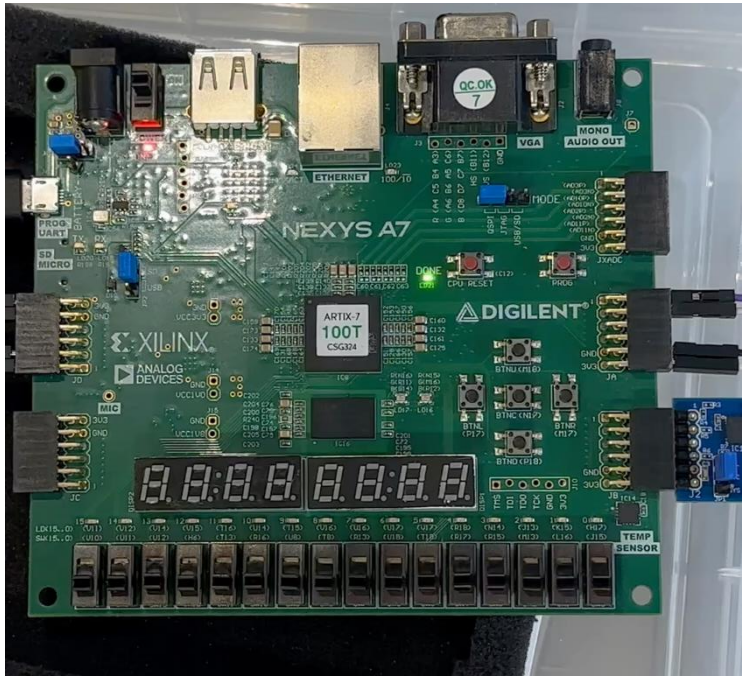


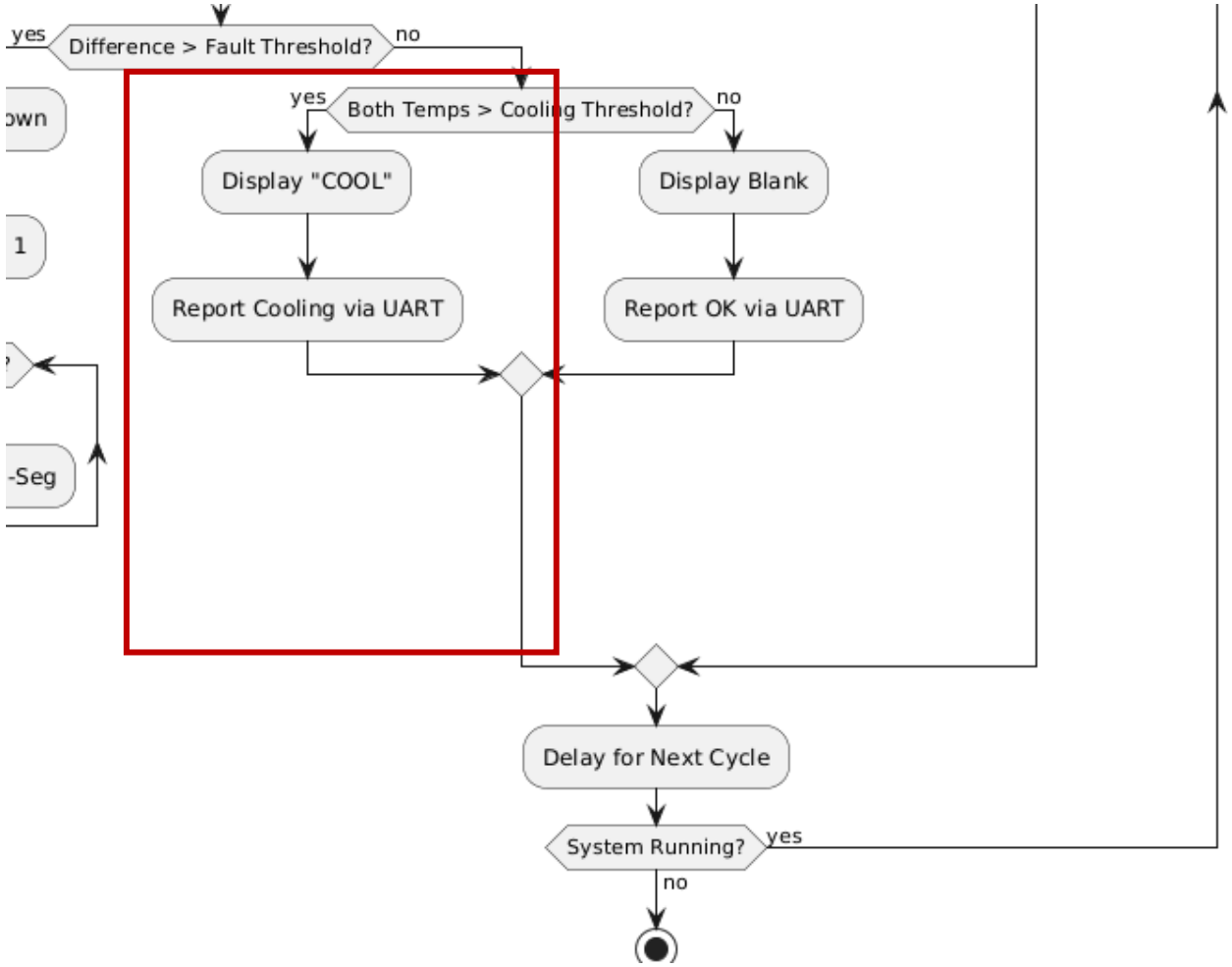
If the difference exceeds a 3 degrees cel, the system immediately enters a safe shutdown state





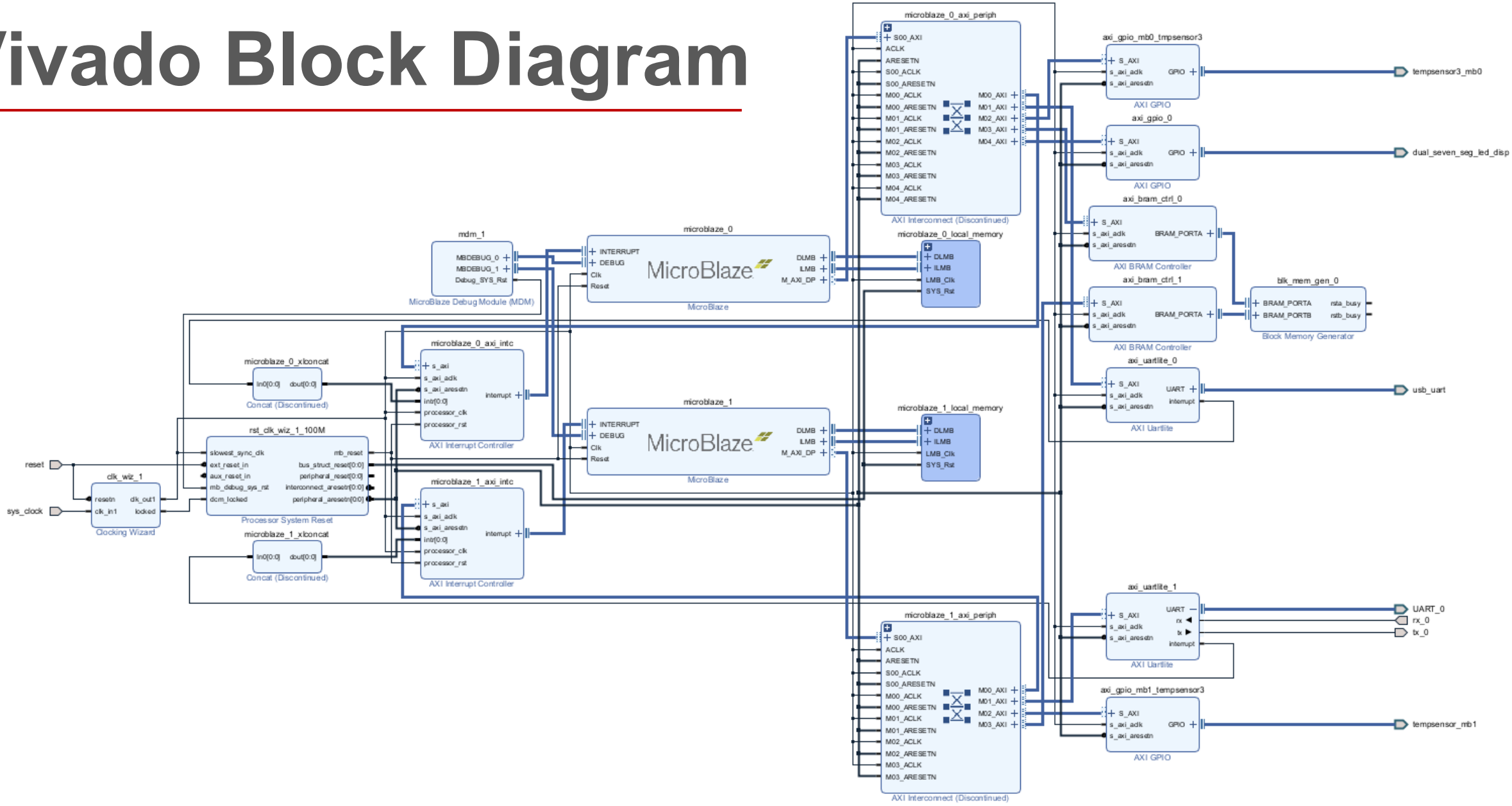
Otherwise, the system either indicates cooling





or continues normal operation

Vivado Block Diagram



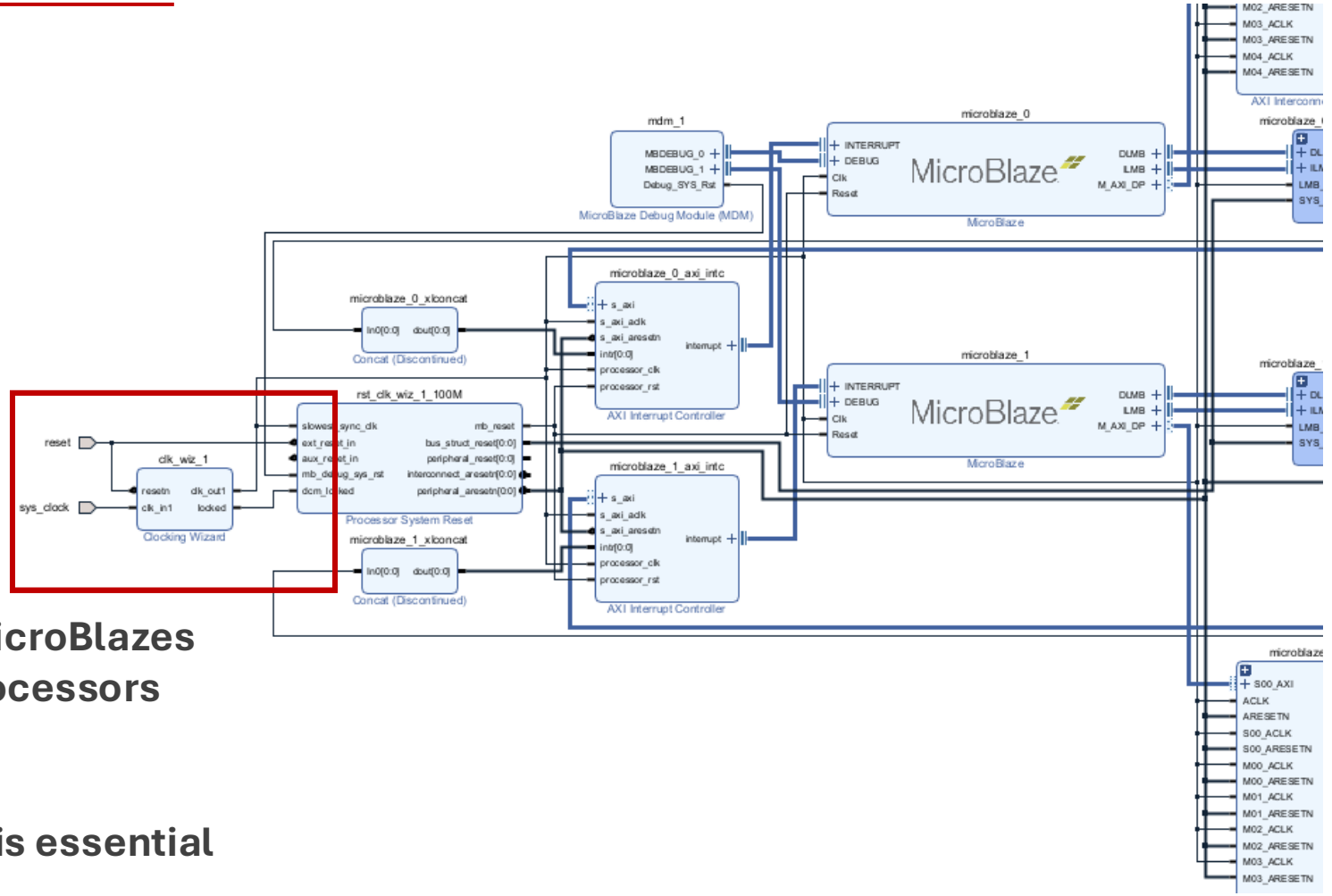
Vivado Block Diagram

One of our main challenges was clock synchronization between the two MicroBlaze processors.

Initially, using separate clock sources caused timing conflicts and prevented successful bitstream generation.

To solve this, we approached the problem systematically and connected both MicroBlazes to a single Clock Wizard, ensuring that both processors operate on the same system clock.

This guarantees deterministic behavior, which is essential for redundancy and safety comparison.

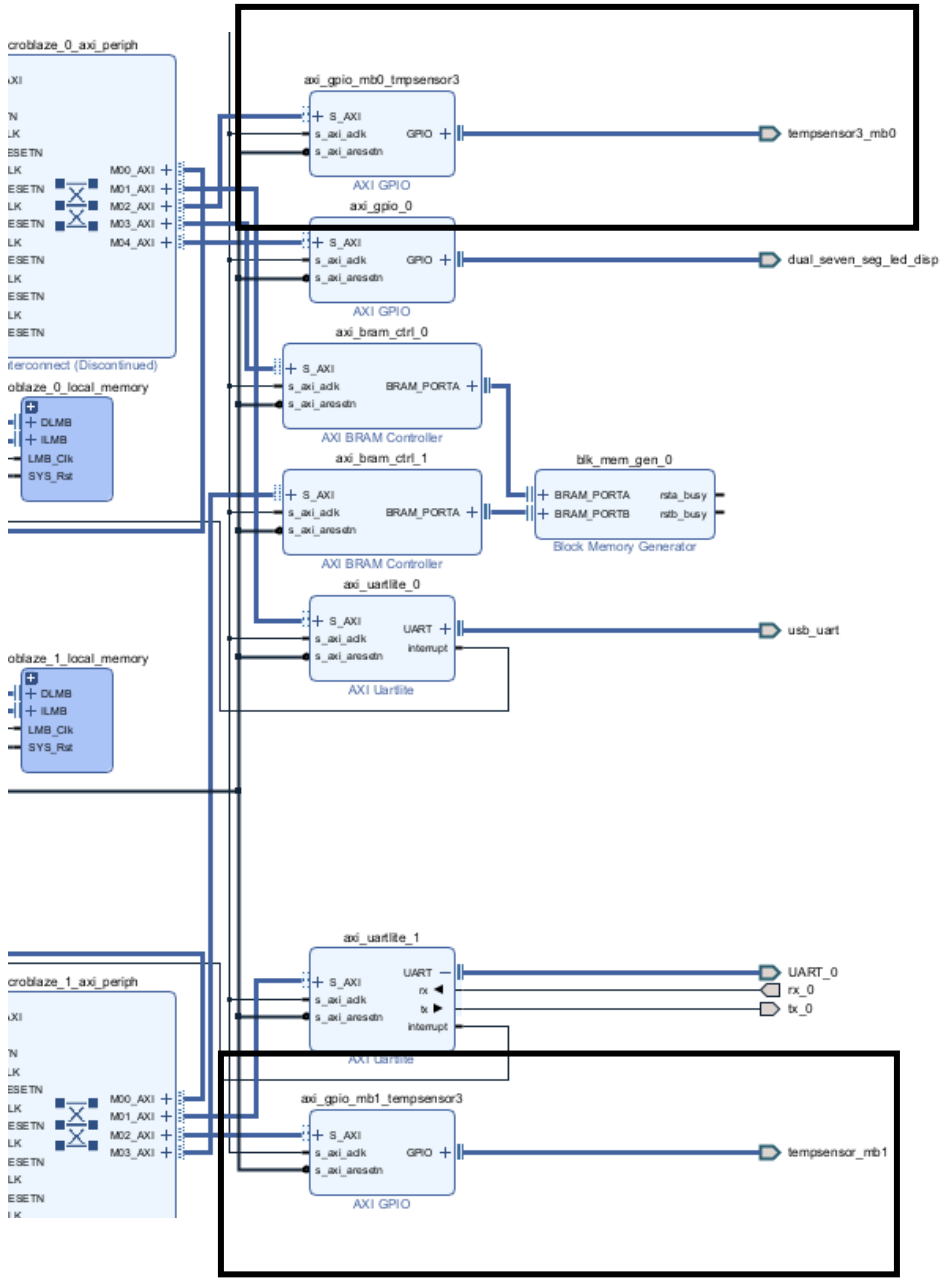


Vivado Block Diagram

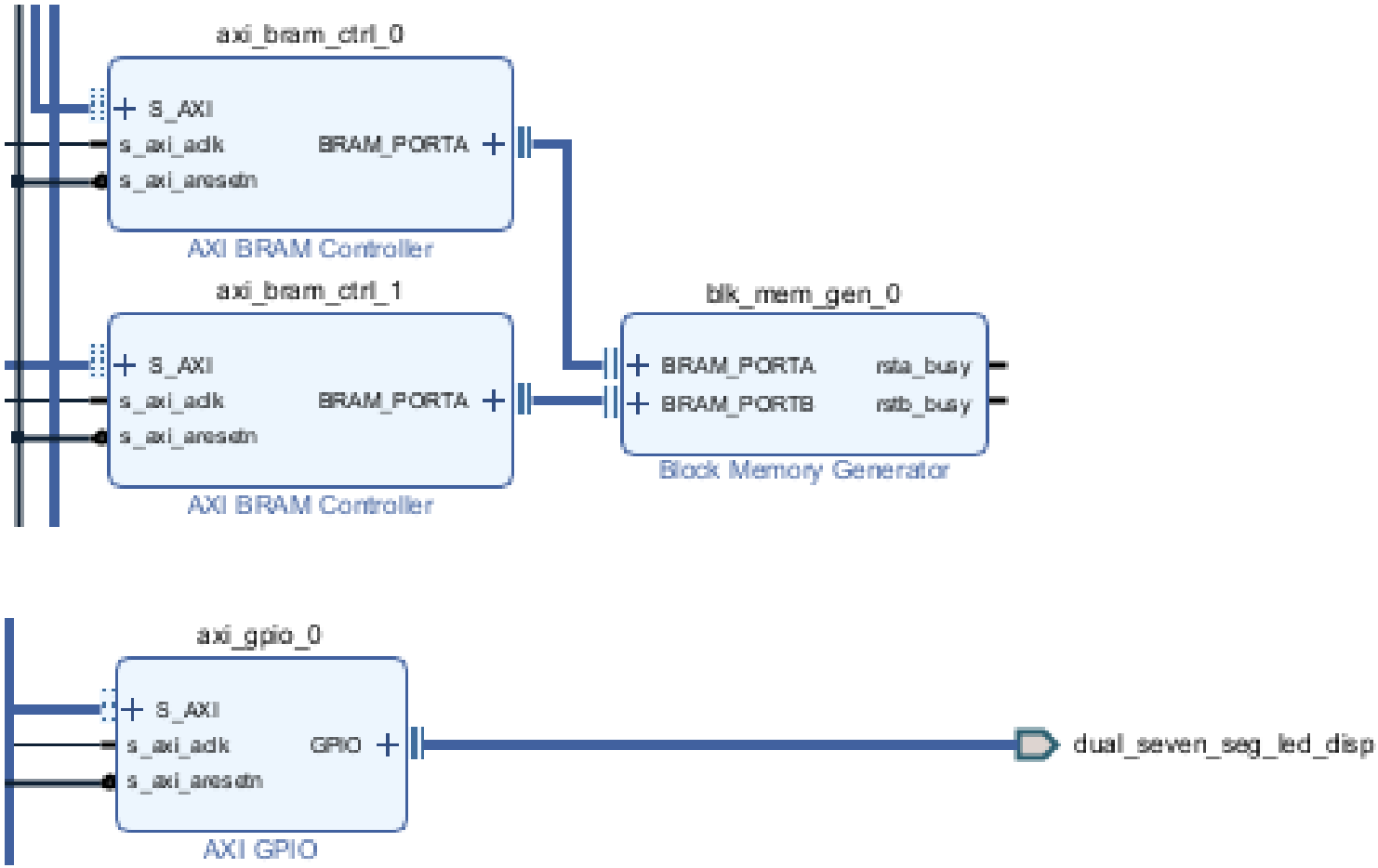
We implemented a bit-banged I²C communication using AXI GPIO.

We configured a GPIO port with two pins, representing the SDA and SCL lines, and implemented the I²C protocol entirely in software.

This allows the MicroBlaze processor to communicate directly with the temperature sensor through the GPIO physical pins, without using a dedicated I²C controller.



Comparator & 7Seg IPs



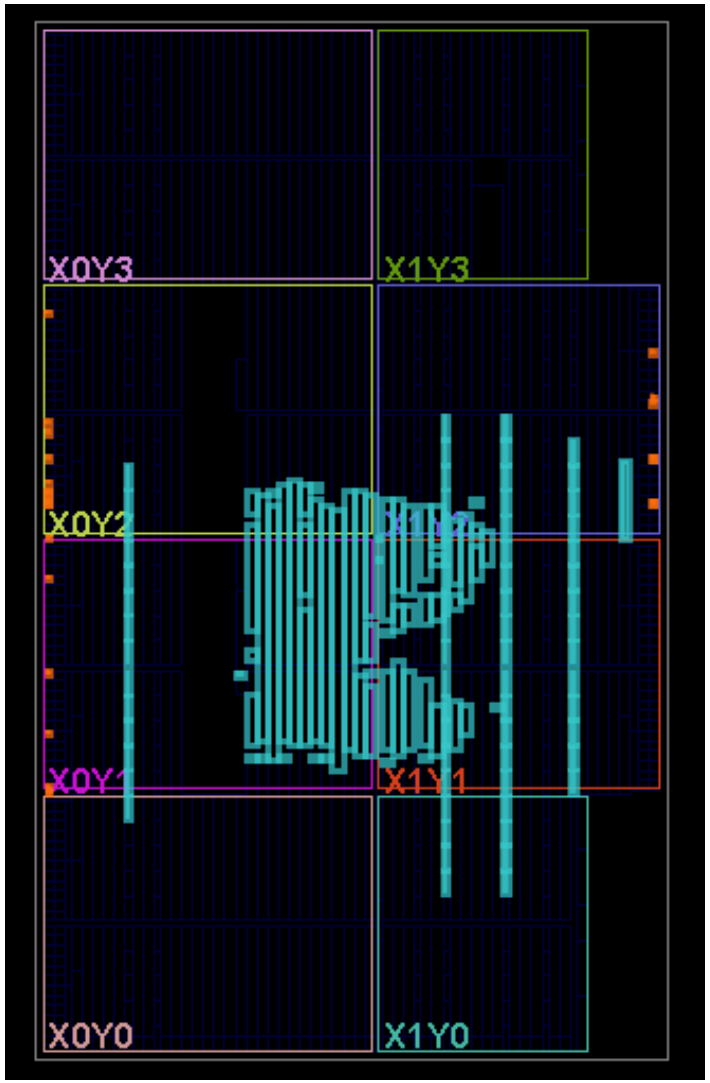
Once communication was established, we implemented a software based comparator using shared Block RAM.

MicroBlaze 1 writes its temperature value into shared memory, and MicroBlaze 0 reads it to perform the safety comparison.

This approach allows flexible threshold control while keeping the hardware design simple.

Comparator & 7Seg IPs

This figure shows the post-synthesis netlist and placement view of our design on the FPGA



Vitis

1 Platform

2 Apps

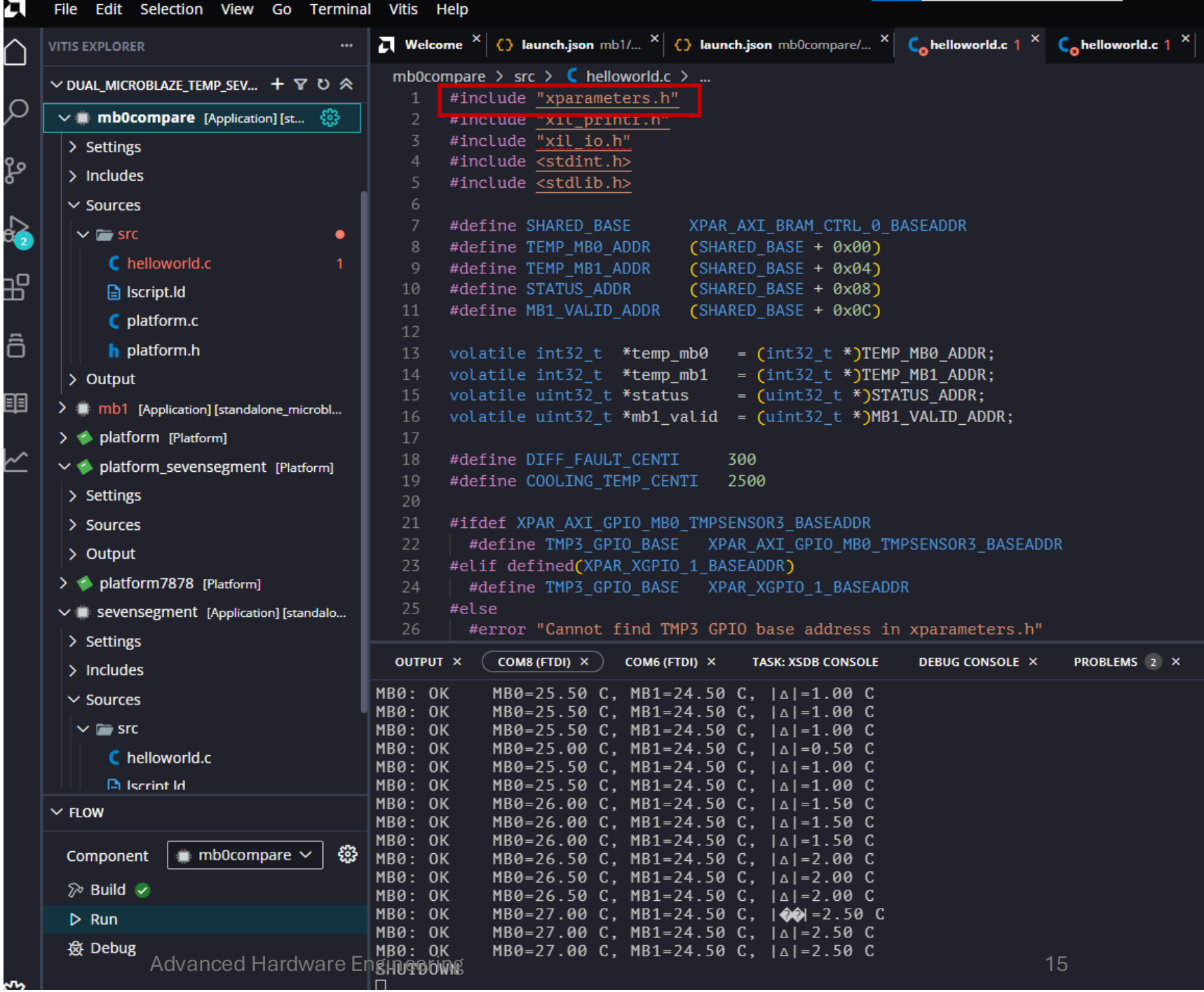
2 Different Domains

2 UARTs

Challenges with:

#include "xparameters.h“
- Search for XPAR

#include “xgpio.h“
- Used xil_io.h



```c
static int sda_read(void)
{
    set_dir(SDA_MASK, 1);
    return (tmp3_data_read() & SDA_MASK) ? 1 : 0;
}

static void i2c_init(void)
{
    tmp3_data_write(0x0);
    set_dir(SCL_MASK | SDA_MASK, 1);
}

static void i2c_start(void)
{
    sda_write(1); scl_write(1); i2c_delay();
    sda_write(0); i2c_delay();
    scl_write(0);
}

static void i2c_stop(void)
{
    sda_write(0); i2c_delay();
    scl_write(1); i2c_delay();
    sda_write(1); i2c_delay();
}

static int i2c_write_byte(uint8_t data)
{
    for (int i = 7; i >= 0; --i) {
        sda_write((data >> i) & 1);
        i2c_delay();
        scl_write(1); i2c_delay();
        scl_write(0);
    }

    sda_write(1);
    i2c_delay();
    scl_write(1); i2c_delay();
    int ack = !sda_read();
    scl_write(0);
    return ack;
}
```

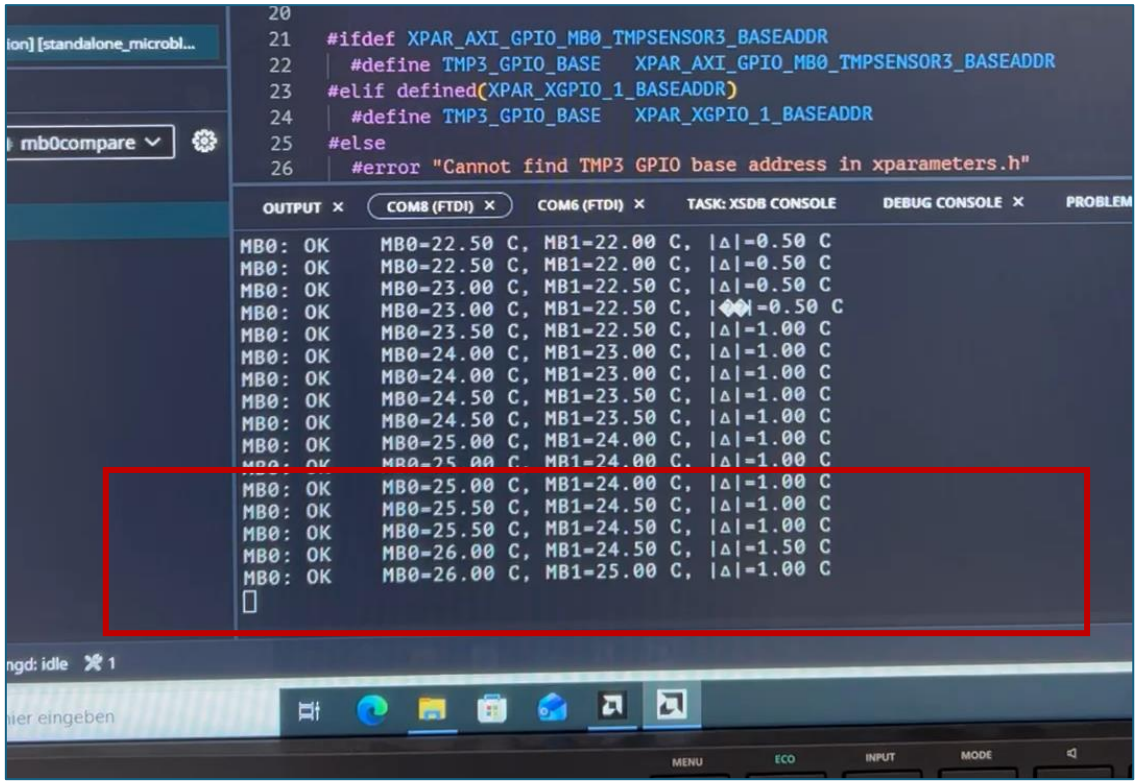
MicroBlaze_0 &1 Codes

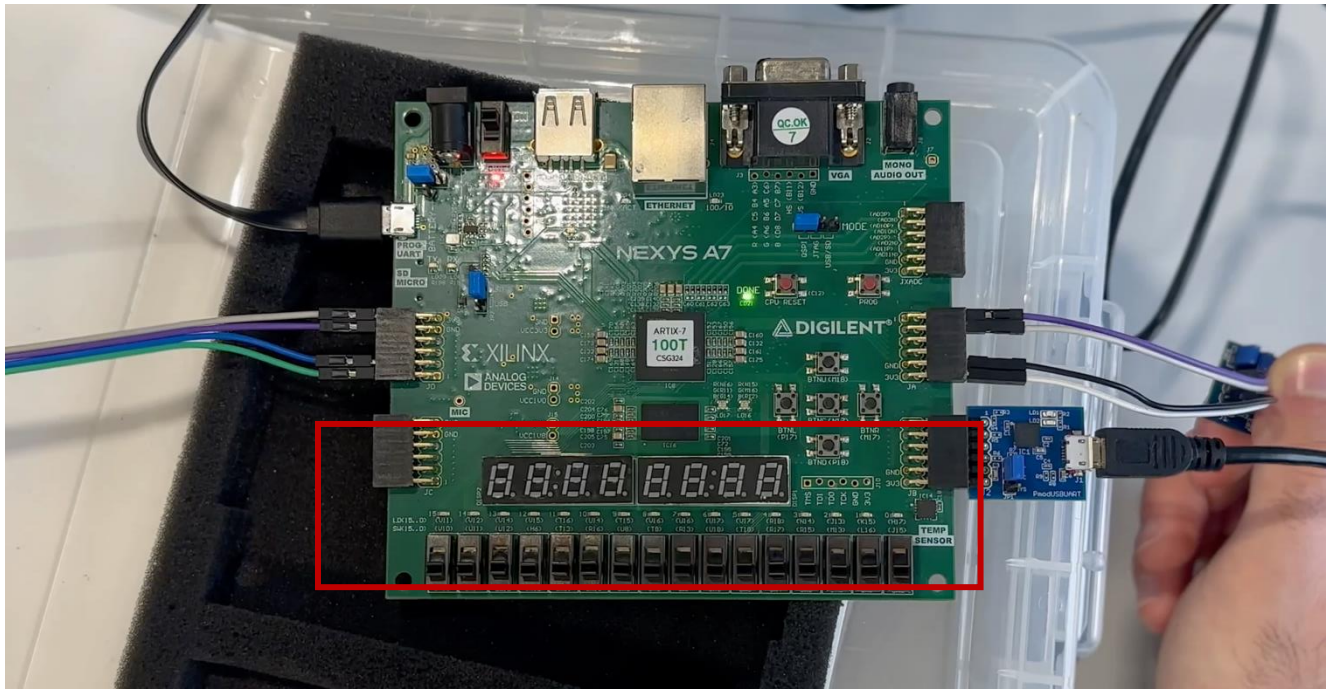
```c
if (*mb1_valid == 0) {
            xil_printf("MB0: MB0=%d.%02d C (waiting for MB1)\r\n", deg0, cent0);
            seg_write(SEG_BLANK);
            delay_cycles(8000000);
            continue;
        }
```

```c
if (diff >= DIFF_FAULT_CENTI) {
            *status = 1;
            xil_printf("SHUTDOWN\r\n");
            *temp_mb0 = t0_centi;
            while (1) {
                seg_show_sequence(MSG_STOP, 4);
            }
        }

        if ((t0_centi > COOLING_TEMP_CENTI) && (t1_centi > COOLING_TEMP_CENTI)) {
            *status = 0;
            xil_printf("comparator: COOL  MB0=%d.%02d C, MB1=%ld.%02ld C\r\n",
                       deg0, cent0, (long)deg1, (long)cent1);
            seg_show_sequence(MSG_COOL, 4);
        } else {
            *status = 0;
            xil_printf("MB0: OK    MB0=%d.%02d C, MB1=%ld.%02ld C, |Δ|=%ld.%02ld C\r\n",
                       deg0, cent0, (long)deg1, (long)cent1,
                       (long)(diff/100), (long)abs(diff%100));
            seg_write(SEG_BLANK);
            delay_cycles(8000000);
        }
```

Live Video COOLING case



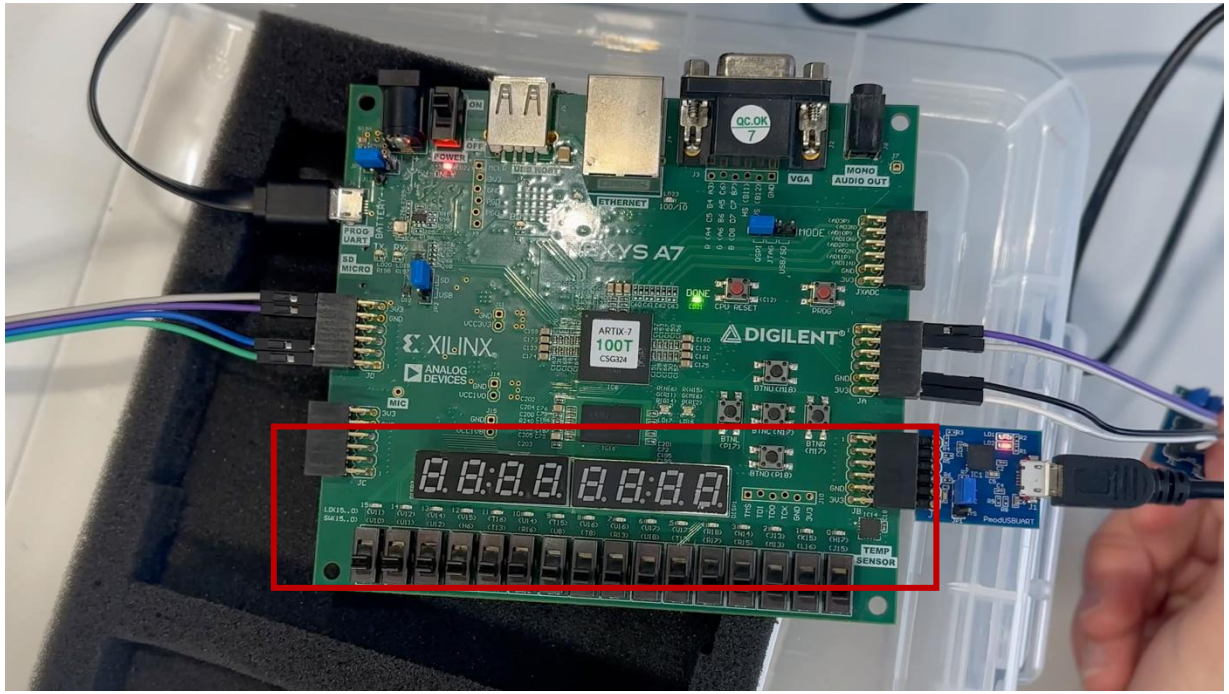


Live Video COOLING case

```
55  #define SEG_T     0x87
56  #define SEG_P     0x8C
57
58  static inline void seg_write(uint8_t pattern)
59  {
```

OUTPUT × COM8 (FTDI) × COM6 (FTDI) × DEBUG CONSOLE × TASK: XSDB C

```
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.00 C, |Δ|=2.00 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
MB0: OK    MB0=23.00 C, MB1=25.50 C, |Δ|=2.50 C
```



Thank You for your Attention
Happy to hear your Questions